SolidityScan | Powered by CRED SHIELDS

Security Assessment Report

Cryptomesh Staking

31 March 2026

This security assessment report was prepared by SolidityScan.com, a cloud-based Smart Contract Scanner.

AICPA SOC

SCS Advocates

OWASP Smart Contract Top 10 Security Risks & Vulnerabilities

Table of Contents

VARIABLES SHOULD BE IMMUTABLE
ARRAY LENGTH CACHING
ASSIGNING TO STRUCTS CAN BE MORE EFFICIENT
AVOID RE-STORING VALUES
CHEAPER CONDITIONAL OPERATORS
CHEAPER INEQUALITIES IN IF()
CHEAPER INEQUALITIES IN REQUIRE()
DEFAULT INT VALUES ARE MANUALLY RESET
DEFINE CONSTRUCTOR AS PAYABLE
GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION
GAS OPTIMIZATION FOR STATE VARIABLES
GAS OPTIMIZATION IN INCREMENTS
LONG NUMBER LITERALS
NAMED RETURN OF LOCAL VARIABLE SAVES GAS AS COMPARED TO RETURN STATEMENT
OPTIMIZING ADDRESS ID MAPPING
SPLITTING REQUIRE STATEMENTS
STORAGE VARIABLE CACHING IN MEMORY
UNNECESSARY CHECKED ARITHMETIC IN LOOP

05 Scan History

06 Disclaimer

01. Vulnerability Classification and Severity

Description

To enhance navigability, the document is organized in descending order of severity for easy reference. Issues are categorized as *Fixed*, *Pending Fix*, or *Won't Fix*, indicating their current status. *Won't Fix* denotes that the team is aware of the issue but has chosen not to resolve it. Issues labeled as *Pending Fix* state that the bug is yet to be resolved. Additionally, each issue's severity is assessed based on the risk of exploitation or the potential for other unexpected or unsafe behavior.

Critical

The issue affects the contract in such a way that funds may be lost, allocated incorrectly, or otherwise result in a significant loss.

High

High-severity vulnerabilities pose a significant risk to both the Smart Contract and the organization. They can lead to user fund losses, may have conditional requirements, and are challenging to exploit.

Medium

The issue affects the ability of the contract to operate in a way that doesn't significantly hinder its behavior.

Low

The issue has minimal impact on the contract's ability to operate.

Informational

The issue does not affect the contract's operational capability but is considered good practice to address.

Gas

This category deals with optimizing code and refactoring to conserve gas.

02. Executive Summary

Cryptomesh Staking

Uploaded Solidity File(s)

Language	Audit Methodology	Website
Solidity	**Static Scanning**	-
Publishers/Owner Name	**Organization**	**Contact Email**
-	-	-

Security Score is AVERAGE

The SolidityScan score is calculated based on lines of code and weights assigned to each issue depending on the severity and confidence. To improve your score, view the detailed result and leverage the remediation solutions provided.

This report has been prepared for Cryptomesh Staking using SolidityScan to scan and discover vulnerabilities and safe coding practices in their smart contract including the libraries used by the contract that are not officially recognized. The SolidityScan tool runs a comprehensive static analysis on the Solidity code and finds vulnerabilities ranging from minor gas optimizations to major vulnerabilities leading to the loss of funds. The coverage scope pays attention to all the informational and critical vulnerabilities with over 700+ modules. The scanning and auditing process covers the following areas:

Various common and uncommon attack vectors will be investigated to ensure that the smart contracts are secure from malicious actors. The scanner modules find and flag issues related to Gas optimizations that help in reducing the overall Gas cost It scans and evaluates the codebase against industry best practices and standards to ensure compliance It makes sure that the officially recognized libraries used in the code are secure and up to date.

The SolidityScan Team recommends running regular audit scans to identify any vulnerabilities that are introduced after Cryptomesh Staking introduces new features or refactors the code.

03. Findings Summary

Cryptomesh Staking
File Scan

Security Score	Scan duration	Lines of code
70.78/100	58 secs	226

0	1	1	4	95	59
Crit	High	Med	Low	Info	Gas

This audit report has not been verified by the SolidityScan team. To learn more about our published reports. click here

ACTION TAKEN

0	0	0	164
Fixed	False Positive	Won't Fix	Pending Fix

S. No.	Severity	Bug Type	Instances	Detection Method	Status
H001	High	IMPROPER VALIDATION IN REQUIRE/ASSERT STATEMENTS	1	Automated	Pending Fix
M001	Medium	LOCKUP PERIOD CAN BE SHORTENED VIA TRANSFERSTAKE, BYPASSING INTENDED LOCK RESTRICTIONS	1	SolidityScan AI	Pending Fix
L001	Low	EVENT BASED REENTRANCY	2	Automated	Pending Fix
L002	Low	USE OF FLOATING PRAGMA	1	Automated	Pending Fix
L003	Low	OUTDATED COMPILER VERSION	1	Automated	Pending Fix
I001	Informational	BLOCK VALUES AS A PROXY FOR TIME	12	Automated	Pending Fix
I002	Informational	CONSIDER USING UINT48 FOR TIME-RELATED VARIABLES	2	Automated	Pending Fix
I003	Informational	IF-STATEMENT REFACTORING	1	Automated	Pending Fix
I004	Informational	MISSING @AUTHOR IN NATSPEC COMMENTS FOR CONTRACT DECLARATION	1	Automated	Pending Fix
I005	Informational	MISSING @DEV IN NATSPEC COMMENTS FOR CONTRACT DECLARATION	1	Automated	Pending Fix
I006	Informational	MISSING @DEV IN NATSPEC COMMENTS FOR FUNCTIONS	17	Automated	Pending Fix
I007	Informational	MISSING @INHERITDOC ON OVERRIDE FUNCTIONS	18	Automated	Pending Fix
I008	Informational	MISSING NATSPEC COMMENTS IN SCOPE BLOCKS	20	Automated	Pending Fix

S. No.	Severity	Bug Type	Instances	Detection Method	Status
I009	Informational	MISSING @NOTICE IN NATSPEC COMMENTS FOR CONSTRUCTORS	1	Automated	*Pending Fix*
I010	Informational	MISSING @NOTICE IN NATSPEC COMMENTS FOR FUNCTIONS	17	Automated	*Pending Fix*
I011	Informational	MISSING @NOTICE IN NATSPEC COMMENTS FOR MODIFIERS	1	Automated	*Pending Fix*
I012	Informational	MISSING UNDERSCORE IN NAMING VARIABLES	2	Automated	*Pending Fix*
I013	Informational	MODIFIER CREATED BUT NEVER USED	1	Automated	*Pending Fix*
I014	Informational	USE CALL INSTEAD OF TRANSFER OR SEND	2	Automated	*Pending Fix*
I015	Informational	VARIABLES SHOULD BE IMMUTABLE	1	Automated	*Pending Fix*
G001	Gas	ARRAY LENGTH CACHING	6	Automated	*Pending Fix*
G002	Gas	ASSIGNING TO STRUCTS CAN BE MORE EFFICIENT	3	Automated	*Pending Fix*
G003	Gas	AVOID RE-STORING VALUES	1	Automated	*Pending Fix*
G004	Gas	CHEAPER CONDITIONAL OPERATORS	1	Automated	*Pending Fix*
G005	Gas	CHEAPER INEQUALITIES IN IF()	2	Automated	*Pending Fix*
G006	Gas	CHEAPER INEQUALITIES IN REQUIRE()	5	Automated	*Pending Fix*
G007	Gas	DEFAULT INT VALUES ARE MANUALLY RESET	4	Automated	*Pending Fix*
G008	Gas	DEFINE CONSTRUCTOR AS PAYABLE	1	Automated	*Pending Fix*
G009	Gas	GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION	1	Automated	*Pending Fix*
G010	Gas	GAS OPTIMIZATION FOR STATE VARIABLES	3	Automated	*Pending Fix*

S. No.	Severity	Bug Type	Instances	Detection Method	Status
G011	Gas	GAS OPTIMIZATION IN INCREMENTS	3	Automated	*Pending Fix*
G012	Gas	LONG NUMBER LITERALS	1	Automated	*Pending Fix*
G013	Gas	NAMED RETURN OF LOCAL VARIABLE SAVES GAS AS COMPARED TO RETURN STATEMENT	4	Automated	*Pending Fix*
G014	Gas	OPTIMIZING ADDRESS ID MAPPING	7	Automated	*Pending Fix*
G015	Gas	SPLITTING REQUIRE STATEMENTS	3	Automated	*Pending Fix*
G016	Gas	STORAGE VARIABLE CACHING IN MEMORY	7	Automated	*Pending Fix*
G017	Gas	UNNECESSARY CHECKED ARITHMETIC IN LOOP	9	Automated	*Pending Fix*

04. Vulnerability Details

Issue Type

IMPROPER VALIDATION IN REQUIRE/ASSERT STATEMENTS

S. No.	Severity	Detection Method	Instances
H001	High	Automated	1

Bug ID	File Location	Line No.	Action Taken
SSP_126589_72	--	--	*Pending Fix*

Upgrade your Plan to view the full report

1 High Issues Found

Please upgrade your plan to view all the issues in your report.

Upgrade

Issue Type

LOCKUP PERIOD CAN BE SHORTENED VIA TRANSFERSTAKE, BYPASSING INTENDED LOCK RESTRICTIONS

S. No.	Severity	Detection Method	Instances
M001	Medium	SolidityScan AI	1

Bug ID	File Location	Line No.	Action Taken
SSP_126589_164	--	--	*Pending Fix*

Upgrade your Plan to view the full report

1 Medium Issues Found

Please upgrade your plan to view all the issues in your report.

Upgrade

Issue Type

EVENT BASED REENTRANCY

S. No.	Severity	Detection Method	Instances
L001	Low	Automated	2

Bug ID	File Location	Line No.	Action Taken
SSP_126589_152	--	--	*Pending Fix*
SSP_126589_153	--	--	*Pending Fix*

Upgrade your Plan to view the full report

2 Low Issues Found

Please upgrade your plan to view all the issues in your report.

Upgrade

Issue Type

BLOCK VALUES AS A PROXY FOR TIME

S. No.	Severity	Detection Method	Instances
I001	Informational	Automated	12

Bug ID	File Location	Line No.	Action Taken
SSP_126589_137	--	--	*Pending Fix*
SSP_126589_137	--	--	*Pending Fix*
SSP_126589_138	--	--	*Pending Fix*
SSP_126589_138	--	--	*Pending Fix*
SSP_126589_139	--	--	*Pending Fix*

Upgrade your Plan to view the full report

12 Informational Issues Found

Please upgrade your plan to view all the issues in your report.

Upgrade

Bug ID	File Location	Line No.	Action Taken
SSP_126589_145	--	--	⚠ *Pending Fix*
SSP_126589_146	--	--	⚠ *Pending Fix*

Upgrade your Plan to view the full report

12 Informational Issues Found

Please upgrade your plan to view all the issues in your report.

Upgrade

Issue Type

ARRAY LENGTH CACHING

S. No.	Severity	Detection Method	Instances
G001	Gas	Automated	6

Description

During each iteration of the loop, reading the length of the array uses more gas than is necessary. In the most favorable scenario, in which the length is read from a memory variable, storing the array length in the stack can save about 3 gas per iteration. In the least favorable scenario, in which external calls are made during each iteration, the amount of gas wasted can be significant.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_57	Cryptomesh.sol	L81 - L88	*Pending Fix*
SSP_126589_58	Cryptomesh.sol	L147 - L155	*Pending Fix*
SSP_126589_59	Cryptomesh.sol	L164 - L168	*Pending Fix*
SSP_126589_60	Cryptomesh.sol	L176 - L181	*Pending Fix*
SSP_126589_61	Cryptomesh.sol	L220 - L224	*Pending Fix*
SSP_126589_62	Cryptomesh.sol	L234 - L249	*Pending Fix*

Issue Type

ASSIGNING TO STRUCTS CAN BE MORE EFFICIENT

S. No.	Severity	Detection Method	Instances
G002	Gas	Automated	3

Description

The contract is found to contain a struct with multiple variables defined in it. When a struct is assigned in a single op eration, Solidity may perform costly storage operations, which can be inefficient. This often results in increased gas costs due to multiple SLOAD and SSTORE operations happening at once

Bug ID	File Location	Line No.	Action Taken
SSP_126589_52	Cryptomesh.sol	L53 - L59	*Pending Fix*
SSP_126589_53	Cryptomesh.sol	L95 - L101	*Pending Fix*
SSP_126589_54	Cryptomesh.sol	L199 - L204	*Pending Fix*

Issue Type

AVOID RE-STORING VALUES

S. No.	Severity	Detection Method	Instances
G003	Gas	Automated	1

Description

The function is found to be allowing re-storing the value in the contract's state variable even when the old value is equal to the new value. This practice results in unnecessary gas consumption due to the `Gsreset` operation (2900 gas), which could be avoided. If the old value and the new value are the same, not updating the storage would avoid this cost and could instead incur a `Gcoldsload` (2100 gas) or a `Gwarmaccess` (100 gas), potentially saving gas.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_154	Cryptomesh.sol	L43 - L70	⚠ *Pending Fix*

Issue Type

CHEAPER CONDITIONAL OPERATORS

S. No.	Severity	Detection Method	Instances
G004	Gas	Automated	1

Description

During compilation, `x != 0` is cheaper than `x > 0` for unsigned integers in solidity inside conditional statements.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_1	Cryptomesh.sol	L127 - L127	*Pending Fix*

Issue Type

CHEAPER INEQUALITIES IN IF()

S. No.	Severity	Detection Method	Instances
G005	Gas	Automated	2

Description

The contract was found to be doing comparisons using inequalities inside the if statement.
When inside the `if` statements, non-strict inequalities `(>=, <=)` are usually cheaper than the strict equalities `(>, <)`.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_76	Cryptomesh.sol	L178 - L178	*Pending Fix*
SSP_126589_77	Cryptomesh.sol	L237 - L237	*Pending Fix*

Issue Type

CHEAPER INEQUALITIES IN REQUIRE()

S. No.	Severity	Detection Method	Instances
G006	Gas	Automated	5

Description

The contract was found to be performing comparisons using inequalities inside the `require` statement. When inside the `require` statements, non-strict inequalities `(>=, <=)` are usually costlier than strict equalities `(>, <)`.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_159	Cryptomesh.sol	L44 - L44	*Pending Fix*
SSP_126589_160	Cryptomesh.sol	L73 - L73	*Pending Fix*
SSP_126589_161	Cryptomesh.sol	L74 - L74	*Pending Fix*
SSP_126589_162	Cryptomesh.sol	L90 - L90	*Pending Fix*
SSP_126589_163	Cryptomesh.sol	L124 - L124	*Pending Fix*

Issue Type

DEFAULT INT VALUES ARE MANUALLY RESET

S. No.	Severity	Detection Method	Instances
G007	Gas	Automated	4

Description

The contract is found to inefficiently reset integer variables to their default value of zero using manual assignment. In Solidity, manually setting a variable to its default value does not free up storage space, leading to unnecessary gas c onsumption. Instead, using the `.delete` keyword can achieve the same result while also freeing up storage space on the Ethereum blockchain, resulting in gas cost savings.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_2	Cryptomesh.sol	L84 - L84	*Pending Fix*
SSP_126589_3	Cryptomesh.sol	L85 - L85	*Pending Fix*
SSP_126589_4	Cryptomesh.sol	L129 - L129	*Pending Fix*
SSP_126589_5	Cryptomesh.sol	L240 - L240	*Pending Fix*

Issue Type

DEFINE CONSTRUCTOR AS PAYABLE

S. No.	Severity	Detection Method	Instances
G008	Gas	Automated	1

Description

Developers can save around 10 opcodes and some gas if the constructors are defined as payable.
However, it should be noted that it comes with risks because payable constructors can accept ETH during deployme
nt.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_65	Cryptomesh.sol	L34 - L36	*Pending Fix*

Issue Type

GAS INEFFICIENCY DUE TO MULTIPLE OPERANDS IN SINGLE IF/ELSEIF CONDITION

S. No.	Severity	Detection Method	Instances
G009	Gas	Automated	1

Description

The contract is found to use multiple operands within a single `if` or `else if` statement, which can lead to unnecessary gas consumption due to the way the EVM evaluates compound boolean expressions. Each operand in a compound condition is evaluated even if the first condition fails, unless short-circuiting occurs, and the combined logic can result in more complex bytecode and higher gas usage compared to using nested `if` statements. This inefficiency is particularly relevant in functions that are called frequently or within loops.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_73	Cryptomesh.sol	L49 - L51	⚠ *Pending Fix*

Issue Type

GAS OPTIMIZATION FOR STATE VARIABLES

S. No.	Severity	Detection Method	Instances
G010	Gas	Automated	3

Description

Plus equals (`+=`) costs more gas than addition operator. The same thing happens with minus equals (`-=`).

Bug ID	File Location	Line No.	Action Taken
SSP_126589_78	Cryptomesh.sol	L61 - L61	*Pending Fix*
SSP_126589_79	Cryptomesh.sol	L103 - L103	*Pending Fix*
SSP_126589_80	Cryptomesh.sol	L130 - L130	*Pending Fix*

Issue Type

GAS OPTIMIZATION IN INCREMENTS

S. No.	Severity	Detection Method	Instances
G011	Gas	Automated	3

Description

`++i` costs less gas compared to `i++` or `i += 1` for unsigned integers. In `i++`, the compiler has to create a temporary variable to store the initial value. This is not the case with `++i` in which the value is directly incremented and returned, thus, making it a cheaper alternative.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_49	Cryptomesh.sol	L198 - L198	*Pending Fix*
SSP_126589_50	Cryptomesh.sol	L220 - L220	*Pending Fix*
SSP_126589_50	Cryptomesh.sol	L234 - L234	*Pending Fix*

Issue Type

LONG NUMBER LITERALS

S. No.	Severity	Detection Method	Instances
G012	Gas	Automated	1

Description

Solidity supports multiple rational and integer literals, including decimal fractions and scientific notations. The use of very large numbers with too many digits was detected in the code that could have been optimized using a different n otation also supported by Solidity.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_31	Cryptomesh.sol	L139 - L139	*Pending Fix*

Issue Type

NAMED RETURN OF LOCAL VARIABLE SAVES GAS AS COMPARED TO RETURN STATEMENT

S. No.	Severity	Detection Method	Instances
G013	Gas	Automated	4

Description

The function having a return type is found to be declaring a local variable for returning, which causes extra gas cons umption. This inefficiency arises because creating and manipulating local variables requires additional computational steps and memory allocation.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_66	Cryptomesh.sol	L136 - L141	*Pending Fix*
SSP_126589_67	Cryptomesh.sol	L162 - L170	*Pending Fix*
SSP_126589_68	Cryptomesh.sol	L194 - L207	*Pending Fix*
SSP_126589_69	Cryptomesh.sol	L217 - L251	*Pending Fix*

Issue Type

OPTIMIZING ADDRESS ID MAPPING

S. No.	Severity	Detection Method	Instances
G014	Gas	Automated	7

Description

Combining multiple address/ID mappings into a single mapping using a struct enhances storage efficiency, simplifies code, and reduces gas costs, resulting in a more streamlined and cost-effective smart contract design.
It saves storage slot for the mapping and depending on the circumstances and sizes of types, it can avoid a Gsset (2 0000 gas) per mapping combined. Reads and subsequent writes can also be cheaper when a function requires both values and they fit in the same storage slot.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_85	Cryptomesh.sol	L19 - L19	*Pending Fix*
SSP_126589_86	Cryptomesh.sol	L20 - L20	*Pending Fix*
SSP_126589_87	Cryptomesh.sol	L21 - L21	*Pending Fix*
SSP_126589_88	Cryptomesh.sol	L22 - L22	*Pending Fix*
SSP_126589_89	Cryptomesh.sol	L24 - L24	*Pending Fix*
SSP_126589_90	Cryptomesh.sol	L25 - L25	*Pending Fix*
SSP_126589_91	Cryptomesh.sol	L27 - L27	*Pending Fix*

Issue Type

SPLITTING REQUIRE STATEMENTS

S. No.	Severity	Detection Method	Instances
G015	Gas	Automated	3

Description

Require statements when combined using operators in a single statement usually lead to a larger deployment gas co st but with each runtime calls, the whole thing ends up being cheaper by some gas units.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_149	Cryptomesh.sol	L44 - L44	*Pending Fix*
SSP_126589_150	Cryptomesh.sol	L73 - L73	*Pending Fix*
SSP_126589_151	Cryptomesh.sol	L74 - L74	*Pending Fix*

Issue Type

STORAGE VARIABLE CACHING IN MEMORY

S. No.	Severity	Detection Method	Instances
G016	Gas	Automated	7

Description

The contract is using the state variable multiple times in the function.
SLOADs are expensive (100 gas after the 1st one) compared to MLOAD / MSTORE (3 gas each).

Bug ID	File Location	Line No.	Action Taken
SSP_126589_24	Cryptomesh.sol	L43 - L70	*Pending Fix*
SSP_126589_25	Cryptomesh.sol	L72 - L107	*Pending Fix*
SSP_126589_26	Cryptomesh.sol	L143 - L160	*Pending Fix*
SSP_126589_27	Cryptomesh.sol	L162 - L170	*Pending Fix*
SSP_126589_28	Cryptomesh.sol	L172 - L184	*Pending Fix*
SSP_126589_29	Cryptomesh.sol	L194 - L207	*Pending Fix*
SSP_126589_30	Cryptomesh.sol	L217 - L251	*Pending Fix*

Issue Type

UNNECESSARY CHECKED ARITHMETIC IN LOOP

S. No.	Severity	Detection Method	Instances
G017	Gas	Automated	9

Description

Increments inside a loop could never overflow due to the fact that the transaction will run out of gas before the varia ble reaches its limits. Therefore, it makes no sense to have checked arithmetic in such a place.

Bug ID	File Location	Line No.	Action Taken
SSP_126589_109	Cryptomesh.sol	L81 - L81	*Pending Fix*
SSP_126589_110	Cryptomesh.sol	L147 - L147	*Pending Fix*
SSP_126589_111	Cryptomesh.sol	L164 - L164	*Pending Fix*
SSP_126589_112	Cryptomesh.sol	L176 - L176	*Pending Fix*
SSP_126589_113	Cryptomesh.sol	L198 - L198	*Pending Fix*
SSP_126589_114	Cryptomesh.sol	L220 - L220	*Pending Fix*
SSP_126589_114	Cryptomesh.sol	L234 - L234	*Pending Fix*
SSP_126589_115	Cryptomesh.sol	L222 - L222	*Pending Fix*
SSP_126589_116	Cryptomesh.sol	L247 - L247	*Pending Fix*

05. Scan History

Critical High Medium Low Informational Gas

No	Date	Security Score	Scan Overview
1.	2026-03-31	**70.78**	0 1 1 4 95 59

06. Disclaimer

The Reports neither endorse nor condemn any specific project or team, nor do they guarantee the security of any specific project. The contents of this report do not, and should not be interpreted as having any bearing on, the economics of tokens, token sales, or any other goods, services, or assets.

The security audit is not meant to replace functional testing done before a software release.

There is no warranty that all possible security issues of a particular smart contract(s) will be found by the tool, i.e., It is not guaranteed that there will not be any further findings based solely on the results of this evaluation.

Emerging technologies such as Smart Contracts and Solidity carry a high level of technical risk and uncertainty. There is no warranty or representation made by this report to any Third Party in regards to the quality of code, the business model or the proprietors of any such business model, or the legal compliance of any business.

In no way should a third party use these reports to make any decisions about buying or selling a token, product, service, or any other asset. It should be noted that this report is not investment advice, is not intended to be relied on as investment advice, and has no endorsement of this project or team. It does not serve as a guarantee as to the project's absolute security.

The assessment provided by SolidityScan is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis. SolidityScan owes no duty to any third party by virtue of publishing these Reports.

As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent manual audits including manual audit and a public bug bounty program to ensure the security of the smart contracts.